

JavaScript Key Learnings — Question 8

OJT Assessment Prep — Prototypal Inheritance, Object.create() & .call()

1. What is Prototypal Inheritance?

Inheritance means one object gets the **properties and methods** of another — without rewriting them.

```
UserCode    → parent/base — has appendNode()
PremiumUser → child — inherits appendNode() AND has its own highlight()
```

```
// PremiumUser can use appendNode() without rewriting it!
```

■ Think of it like: *PremiumUser IS a UserNode, but with extra features.*

2. Constructor Functions — The Parent (UserCode)

The **parent constructor** sets up the base properties every user needs:

```
function UserNode(username) {
  this.element = document.createElement("li"); // create list item
  this.element.innerText = username;           // set the name
}

// Method goes on prototype — shared by all UserNodes
UserCode.prototype.appendNode = function(ID) {
  let ul = document.getElementById(ID);
  ul.append(this.element);
};
```

3. Inheriting Properties with .call()

The child constructor uses **.call()** to run the parent constructor on its own **this** — copying all properties:

■ Without .call() — child has no this.element:

```
function PremiumUser(username) {
  // this.element is undefined! highlight() will crash
}
```

■ With .call() — parent sets up this.element on child:

```
function PremiumUser(username) {
  UserNode.call(this, username);
  // ↑ runs UserNode constructor BUT with PremiumUser's "this"
  // ↑ so this.element gets created on PremiumUser too!
}
```

Think of it as: *Hey UserNode, run your setup but on MY object!*

4. Object.create() — Linking Prototype Chains

Object.create() creates a new object whose prototype is set to whatever you pass in. Used to link child prototype to parent prototype.

■ **Without Object.create() — appendNode not inherited:**

```
PremiumUser.prototype = {}; // empty – no link to UserNode
new PremiumUser("Alice").appendNode(); // TypeError!
```

■ **With Object.create() — full method inheritance:**

```
PremiumUser.prototype = Object.create(UserNode.prototype);
// Now PremiumUser.prototype is linked to UserNode.prototype
// appendNode() is accessible via the chain!
```

5. The Prototype Chain — How JS Finds Methods

When you call a method, JS searches up the chain until it finds it:

```
newPremiumUser.appendNode("user-list")
```

JS searches:

1. newPremiumUser object itself → not found
2. PremiumUser.prototype → not found (only highlight here)
3. UserNode.prototype → FOUND! appendNode() ■

```
newPremiumUser.highlight()
```

JS searches:

1. newPremiumUser object itself → not found
2. PremiumUser.prototype → FOUND! highlight() ■

6. Adding Child-Only Methods

After linking prototypes, add methods **only the child** should have:

```
PremiumUser.prototype.highlight = function() {
  this.element.style.backgroundColor = "gold";
};
```

```
// Only PremiumUser has highlight()
// UserNode does NOT get this method
```

7. .call() vs Object.create() — The Critical Difference

	<code>.call()</code>	<code>Object.create()</code>
Purpose	Inherit PROPERTIES	Inherit METHODS
What it copies	this.element, this.username etc	appendNode, and other prototype methods
Where used	Inside child constructor	Outside, after constructor defined
Without it	Child has no properties	Child has no inherited methods

■ You need BOTH for full inheritance!

■■ Complete Solution

```
// 1. Parent constructor
function UserNode(username) {
  this.element = document.createElement("li");
  this.element.innerText = username;
}

// 2. Parent method on prototype
UserNode.prototype.appendNode = function(ID) {
  let ul = document.getElementById(ID);
  ul.append(this.element);
};

// 3. Child constructor – inherits properties
function PremiumUser(username) {
  UserNode.call(this, username); // copies this.element
}

// 4. Link prototypes – inherits methods
PremiumUser.prototype = Object.create(UserNode.prototype);

// 5. Child-only method
PremiumUser.prototype.highlight = function() {
  this.element.style.backgroundColor = "gold";
};

// 6. Use them
let alice = new PremiumUser("Alice_VIP");
alice.highlight(); // gold background
alice.appendNode("user-list"); // inherited from UserNode!

let bob = new UserNode("Bob_Normal");
bob.appendNode("user-list"); // normal, no highlight
```

■ Golden Rules to Remember

Rule	Remember
Inherit properties	Use ParentConstructor.call(this, args) inside child constructor
Inherit methods	Use Child.prototype = Object.create(Parent.prototype)
Child-only methods	Add to Child.prototype AFTER Object.create()
Prototype chain	JS searches: instance → child prototype → parent prototype → Object
Pass ID to method	appendNode needs the ul ID — always pass it: appendNode("user-list")